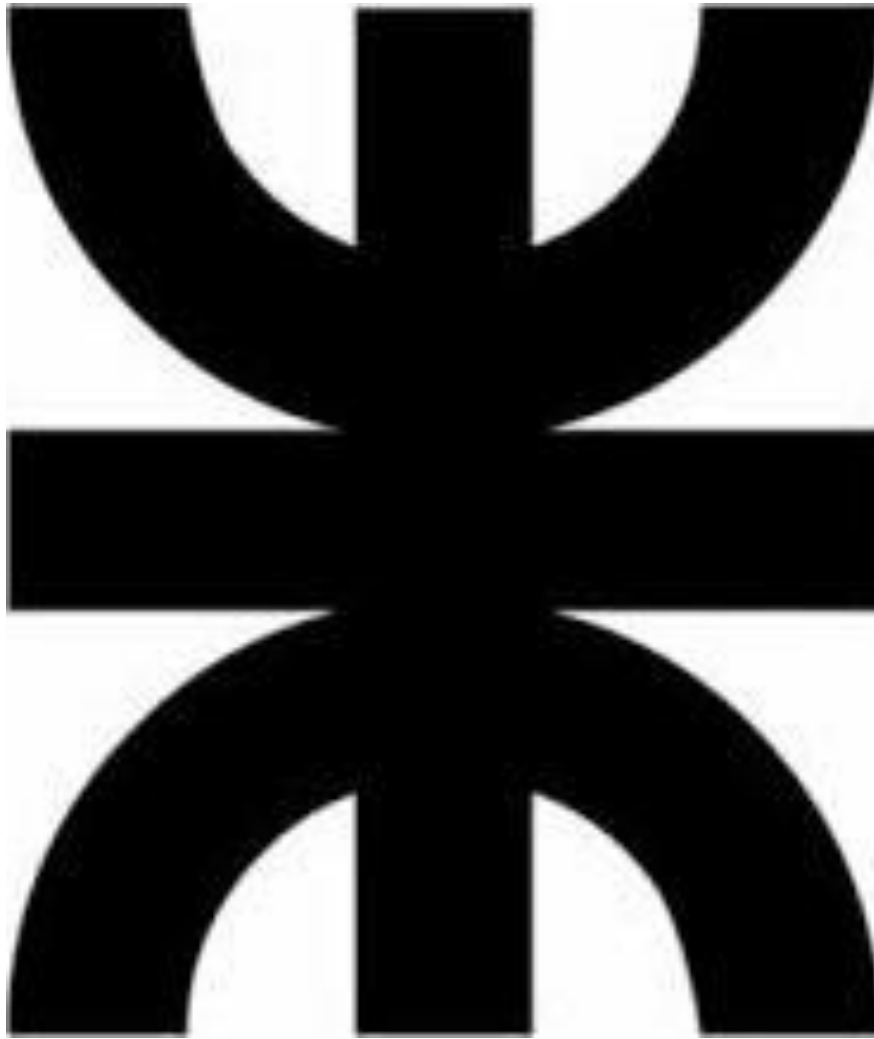




UTN – FRSF
ANÁLISIS NUMÉRICO
2026

INTEGRANTES:

MAURICIO ACOSTA

FEDERICO BLEUEL

MARCOS BUDE

PABLO DELLACQUA

Índice para trabajo práctico 3, Algoritmos para encontrar raíces en funciones no lineales:	
Introducción:.....	3
Reconocimiento de métodos existentes	3
Newton-Raphson (archivo NR.py):.....	3
Bisección (archivo biseccion.py):.....	5
Pequeños ajustes	6
Probando nuevos métodos	7
Interpolación Cuadrática Inversa (archivo metodoIQI.py):.....	7
Regula Falsi (archivo metodoRegulaFalsi.py):.....	9
Secante (archivo metodoSecante.py):	11
Brent-Dekker (archivo metodoBrent.py):.....	13
Método Híbrido versión 1 (archivo metodoHibridoV01.py):.....	15
Método Híbrido Version 2 (archivo metodoHibridoV02.py):.....	18
Resultados:	21
Método V01:.....	21
Método V02:.....	21
Método Secante:.....	22
Método Regula Falsi:	23
Método IQI:	24
Método Brent:	25
Método Híbrido V01:.....	26
Método Híbrido V02:.....	26
Conclusión:	27

Introducción:

El objetivo del TP3 es diseñar e implementar en Python un algoritmo híbrido de búsqueda de raíces de funciones no lineales, capaz de encontrar las soluciones para expresiones complejas (notablemente no lineales, oscilatorias, con raíces múltiples o asíntotas). Como parte de la consigna, no se permite el uso de funciones nativas de optimización del lenguaje de programación, ni paquetes externos como **numpy**, **scipy** u otros. Pueden emplearse las funciones matemáticas incluidas en el módulo **math**. El trabajo limitará su alcance al campo de los números reales. Para asegurar el desarrollo y control de versiones, todo el código y la documentación del proyecto se gestionaron a través del siguiente repositorio de GitHub:

<https://github.com/pabloDLQ/Analisis-numerico-2026>

Los distintos métodos probados están en la carpeta /versiones, mientras que los dados por la cátedra están en la carpeta /data.

Reconocimiento de métodos existentes

La idea de estas primeras versiones fue interpretar y entender los métodos dados por la cátedra.

Comenzamos colocando las funciones en Geogebra para conocer las raíces de manera exacta y así tener una referencia para comparar con las raíces calculadas con los métodos proporcionados por la cátedra (Newton-Raphson y Bisección).

Newton-Raphson (archivo NR.py):

Tipo de Método: Se trata de un método abierto que utiliza un punto inicial y calcula la pendiente (derivada) para trazar una recta tangente y encontrar el corte con el eje x .

Inicialización: El algoritmo toma el punto medio del intervalo indicado $[a,b]$ como su aproximación inicial calculando $x = \frac{a+b}{2}$. Define un incremento minúsculo ($dx = 1e-6$) y establece un límite de 150 iteraciones ($max_iter = 150$) para calcular la derivada.

Evaluación de la función: En cada ciclo, primero se realiza una exploración común evaluando la función en x e informando que es del tipo común para cumplir con el sistema de evaluación de puntos, si coincide con una raíz exacta ($f(x) = 0$) el algoritmo corta y retorna ese valor de x .

Cálculo de la derivada: Se evalúa la función un paso más adelante (con $x + dx$) declarando que es del tipo derivada. Luego se aplica la fórmula para obtener una derivada aproximada ($dfx = fx_derecha - fx / dx$).

Zonas planas: Matemáticamente, si la derivada es cero, ocurre una división por cero que haría colapsar el programa. Para evitar esto se implementó que si el valor absoluto de la derivada calculada es menor a $1e-11$, se esquivo la división por cero moviendo x hacia la mitad derecha o izquierda del intervalo (dependiendo del signo de $f(x)$) y se salta a la siguiente iteración.

Paso de Newton: Si la pendiente es válida, se aplica la fórmula de Newton para calcular el candidato a ser la raíz ($x_{siguiente} = x - f(x) / df(x)$).

Guardián del Confinamiento: Si la curva tiene curvas complejas o es deliberadamente no lineal, el método de Newton puede disparar el siguiente punto hacia el infinito. Para evitar esto el código verifica si $x_{siguiente}$ sigue perteneciendo al intervalo (a,b) , si no pertenece se descarta ese resultado absurdo y se aplica una reestabilización geométrica forzando que el punto vuelva al centro exacto del intervalo ($x_{siguiente} = (a+b) / 2$).

Criterio de Parada (convergencia): Evalúa la distancia entre el punto viejo y el nuevo, si el valor absoluto entre $x_{siguiente}$ y x es menor o igual a la tolerancia (tol), asume que ya no se va a mover mucho más, por lo que detiene el algoritmo y devuelve $x_{siguiente}$ como la raíz.

El funcionamiento de NR.py también se puede representar a través del siguiente diagrama de flujo:

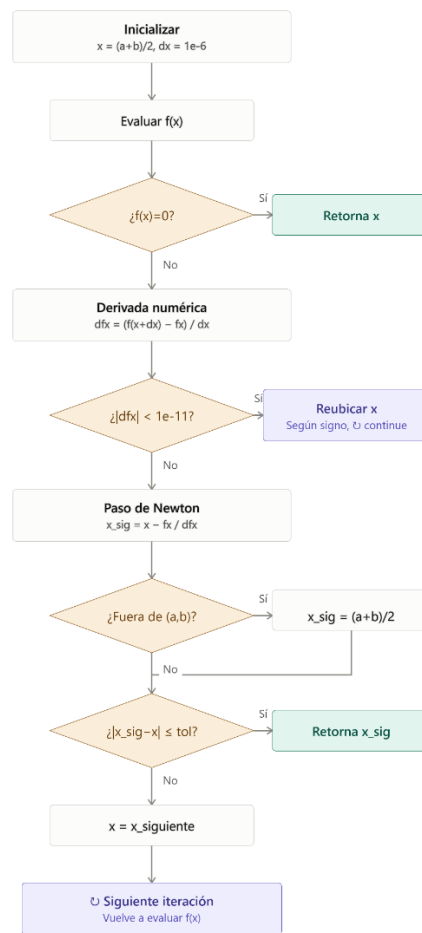


Ilustración 1: Diagrama de flujo de NR.

Bisección (archivo biseccion.py):

Tipo de Método: Se trata de un método cerrado que depende exclusivamente de dividir el intervalo a la mitad en cada iteración, conservando siempre la mitad que encierra el cambio de signo.

Verificación inicial obligatoria: Antes de comenzar, el programa comprueba si el intervalo inicial tiene sentido matemático. Si los extremos tienen el mismo signo, es decir $f(a) \cdot f(b) > 0$, el algoritmo aborta la ejecución y arroja un error (ValueError) ya que no hay garantías de encontrar la raíz.

Salvaguardas de extremos: Si al evaluar la función en los extremos iniciales el resultado da exactamente cero, el algoritmo deja de buscar y devuelve ese punto como la raíz definitiva, ahorrando cómputo inútil.

Convergencia por tamaño: A diferencia de otros métodos que miden la distancia entre iteraciones, el bucle principal de este código está controlado puramente por la dimensión espacial del intervalo. El ciclo continuará

ejecutándose única y exclusivamente mientras la distancia entre b y a sea mayor que la tolerancia ($b - a > tol$).

El funcionamiento de biseccion.py también se puede representar a través del siguiente diagrama de flujo:

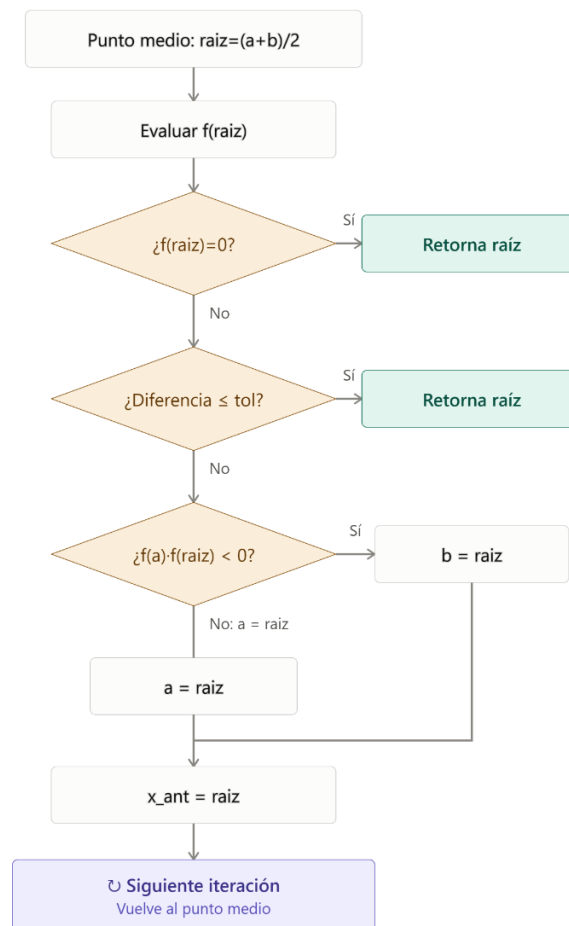


Ilustración 2: Diagrama de flujo de bisección.

Encontramos errores en algunos resultados, por ejemplo, en la función #1 la raíz se encuentra en un punto donde la función es positiva durante todo el intervalo, debido a esto el método de bisección (biseccion.py) arroja una excepción y esto se considera colapso de sistema. Por otro lado, había casos en los que las raíces calculadas por el programa escapaban de la tolerancia de referencia ($1e-7$).

Pequeños ajustes

Ya conocíamos las raíces reales gracias a Geogebra, con eso y las raíces calculadas en la versión 1 comparamos y calculamos la diferencia para mostrar (si correspondía) una penalización de Raíz Incorrecta (Falso Positivo), de esta manera comienza también la implementación del sistema de puntuación,

penalizaciones y multas, siguiendo las mismas reglas declaradas en la consigna (Llamada Común, Llamada de Derivada, Impuesto a la Ráfaga Monótona, Reinicio Cruzado; Aproximación Exitosa, Raíz Incorrecta (Falso Positivo), Colapso del Sistema).

Se corrigió el caso en el que una función tiene el mismo signo durante todo su intervalo de prueba, si ocurre esto se procede a estimar la raíz directamente con NR.py.

Se implementó un método híbrido que mezclaba los dos con una implementación de la tolerancia tal que esta sea un treintaidosavo de la original, para que busque una zona más pequeña en la que luego aplica el método de Newton, pero luego nos corrigieron que esto modificaría el funcionamiento del método bisección, por lo que no es correcto hacerlo, pero lo conservamos a modo de registrar la evolución del proyecto.

Probando nuevos métodos

Una vez entendidos y medidos los métodos anteriores procedimos a investigar nuevas alternativas para encontrar una solución más eficiente. Entre ellas, encontramos los siguientes:

- Método de la Secante (Abierto)
- Método de la Falsa Posición o Regula Falsi (Cerrado)
- Interpolación Cuadrática Inversa (IQI)
- Método de Steffensen
- Método de Muller
- Método de Ridders
- Método Dekker
- Método de Brent

Probamos algunos de los métodos encontrados, comenzando con los métodos no combinados:

Interpolación Cuadrática Inversa (archivo metodoIQI.py):

Tipo de Método: Se trata de un método abierto que utiliza tres puntos iniciales para trazar una parábola invertida, por lo que no depende de que exista un cambio de signo en el intervalo para iterar.

Inicialización: El algoritmo exige tres puntos de arranque para poder trazar la parábola, para ello toma los extremos del intervalo inicial a y b , y calcula el punto medio exacto $(\frac{a+b}{2})$ y se evalúa la función de forma estándar (tipo="comun") en estos tres puntos. Si alguno de ellos es una raíz exacta el método retorna ese valor y el programa finaliza.

Bucle principal y Ordenamiento: Los tres puntos y sus imágenes se guardan en una lista llamada puntos. Al inicio de cada iteración, el código ordena esta lista de mayor a menor tomando como criterio el valor absoluto de la función ($\text{abs}(\text{item}[1])$). Esto garantiza que el elemento guardado en x_0 sea siempre el candidato más cercano a 0.

Comprobación de Parada Inicial: Antes de calcular se verifica si la distancia entre el mejor punto (x_0) y el segundo mejor (x_1) es menor o igual a la tolerancia (tol). En ese caso se termina el proceso y el programa retorna x_0 .

Paso IQI: Para aplicar la fórmula matemática completa de la interpolación, el código debe asegurarse de que los tres valores de la función (f_0, f_1, f_2) sean numéricamente distintos. Si lo son y uno de sus denominadores no es cero, calcula la parábola invertida y obtiene un nuevo punto x_3 .

Control de estabilidad: Si la interpolación cuadrática falla matemáticamente o si el resultado calculó un x_3 que es igual que uno de los puntos que ya se tenían, el algoritmo abandona temporalmente la fórmula IQI y recurre a un paso alternativo (equivalente al método de la Secante o a un promedio simple de los puntos) para forzar al algoritmo a moverse a un lugar nuevo y no estancarse.

Evaluación del candidato y actualización: Se evalúa el nuevo punto x_3 para obtener f_3 (tipo="comun"). Si la imagen es cero o si la distancia respecto a x_0 satisface la tolerancia, el programa retorna x_3 . Caso contrario, descarta el peor de los tres puntos de la lista ($\text{puntos}[2]$) y lo reemplaza por el nuevo par (x_3, f_3) para repetir el proceso.

Falla explícita: Si el algoritmo llega al máximo de iteraciones antes de alcanzar la convergencia exigida por la tolerancia, se lanza una excepción severa (RuntimeError ("No se alcanzó la tolerancia en el máximo de iteraciones")) con el fin de seguir con otra estrategia.

El funcionamiento de metodoIQI.py también se puede representar a través del siguiente diagrama de flujo:

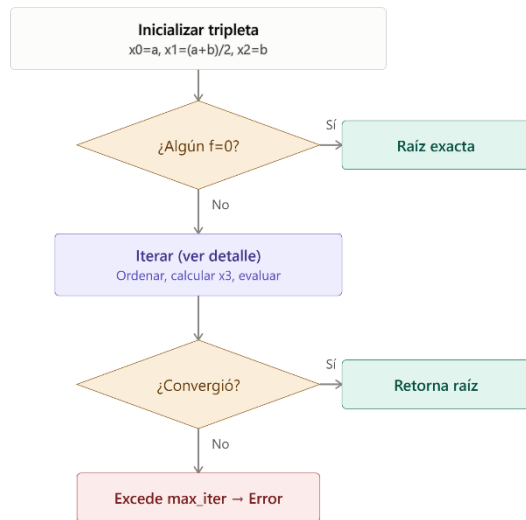


Ilustración 4: Diagrama de flujo inicialización IQI.

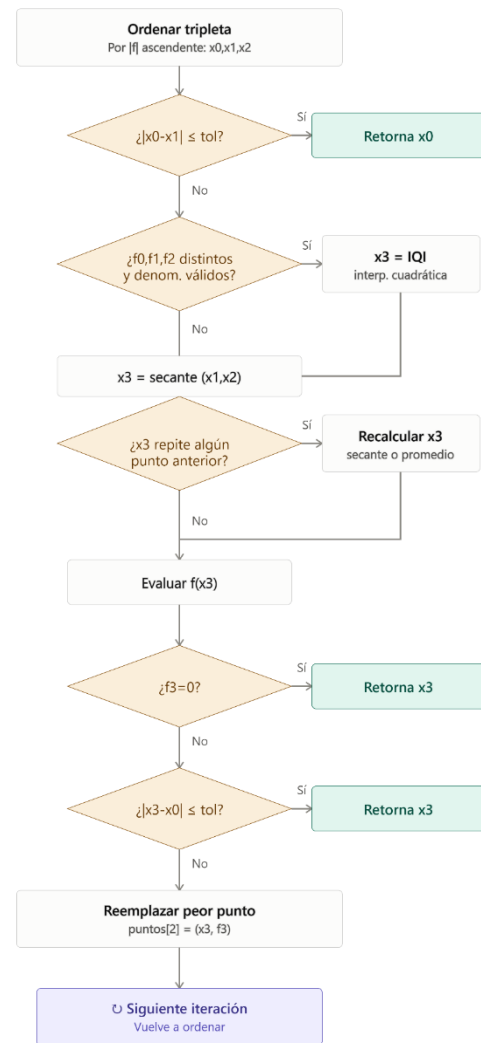


Ilustración 3: Diagrama de flujo iteración IQI.

Regula Falsi (archivo metodoRegulaFalsi.py):

Tipo de Método: Es un método cerrado ya que requiere un encierro previo de la raíz. Traza una recta entre los dos extremos del intervalo y conserva siempre el subintervalo que sigue encerrando la solución tras cada evolución.

Verificación Inicial: Primero, se evalúa la función en los extremos a y b . Si alguno devuelve cero, se retorna ese valor de x . Luego se comprueba si no hay cambio de signo ($f a \cdot f b > 0$), en este caso el método lanza una excepción ValueError porque matemáticamente no puede iniciar.

El paso Regula Falsi: Dentro de un bucle se calcula el denominador de la fórmula ($fb - fa$). Si este denominador es cero, se aborta la ejecución para no generar un error matemático, de lo contrario se calcula el cruce con el eje X (punto c) utilizando la fórmula de interpolación.

Parada por Convergencia: Se evalúa el punto c obteniendo fc , si es igual a cero el programa retorna c como raíz exacta. Además se evalúa el criterio de parada: si la distancia absoluta entre el punto nuevo c y el de la iteración anterior ($x_anterior$) es menor o igual a la tolerancia (tol) se detiene el bucle y retorna la estimación.

Actualización segura del Intervalo: Al igual que en Bisección, se verifica si el cambio de signo ocurre entre fa y fc ($fa \cdot fc < 0$). Si es así, el extremo derecho b toma el lugar de c , de lo contrario significa que la raíz está del otro lado, por lo que el extremo izquierdo a toma el lugar de c . De este modo, se conserva el subintervalo válido.

Aplicación de la Corrección Illinois: Durante la actualización, el algoritmo rastrea cuál de los dos lados se movió la última vez ($lado_anterior$). Si un mismo lado se está actualizando por segunda vez consecutiva significa que el extremo opuesto está “bloqueado”, como consecuencia y sólo si el parámetro opcional `use_illinois` está activado (lo está por defecto) se multiplica el valor de la función del lado bloqueado por 0,5.

Falla explícita por tiempo: Si el ciclo `for` cumple las 150 iteraciones y no logra satisfacer la tolerancia, en lugar de devolver un valor falso e impreciso, se termina la ejecución lanzando una excepción `RuntimeError` dando lugar a otro método de respaldo.

El funcionamiento de `metodoRegulaFalsi.py` también se puede representar a través del siguiente diagrama de flujo:

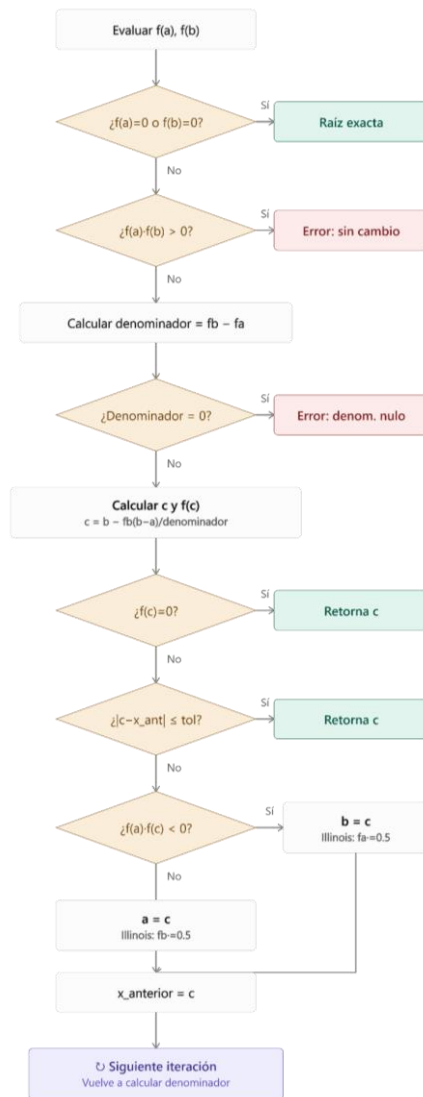


Ilustración 5: Diagrama de flujo de regla falsi.

Secante (archivo metodoSecante.py):

Tipo de Método: Es explícitamente un método abierto que utiliza los dos últimos puntos evaluados para trazar una recta secante, sin garantizar el encierro riguroso de la raíz.

Inicialización y Selección de Semillas: El algoritmo requiere dos puntos iniciales ($x_{anterior}$ y x_{actual}). El código base inicia directamente utilizando los extremos del intervalo a y b . Sin embargo, optamos por una variante más robusta que realiza un “barrido inicial” dividiendo el intervalo en 64 partes. Esto permite evaluar la función y elegir el par adyacente más prometedor para usarlos como semillas iniciales estables antes de comenzar la iteración.

Bucle principal y cálculo: Dentro del límite de iteraciones, el algoritmo calcula el denominador de la fórmula ($f_{actual} - f_{anterior}$) y se aplica la fórmula clásica de la secante utilizando únicamente los dos últimos puntos.

Salvaguardas (Denominador Nulo): Si el denominador es cero (zona plana que causaría división por cero) el código evita el colapso calculando un punto medio basado en los extremos fijos del intervalo original.

Salvaguardas (Confinamiento/Saltos inválidos): Si el cálculo de $x_{siguiente}$ resulta como un valor fuera de los límites del intervalo original $[a,b]$ o arroja un valor no finito, la función rechaza el salto y se realiza un restablecimiento geométrico en el centro del intervalo $(\frac{a+b}{2})$.

Criterio de Parada: Una vez obtenido y validado el nuevo candidato para raíz, lo evalúa $f_{siguiente}$. Si el valor es exactamente 0, el programa retorna la raíz. De lo contrario el programa se detiene estrictamente si la distancia absoluta entre el punto nuevo y el actual es menor o igual a la tolerancia exigida ($|x_{siguiente} - x_{actual}| \leq tol$).

Actualización de la Secante: Si no se alcanzó la tolerancia, el algoritmo “sigue”: El punto que era x_{actual} pasa a ser $x_{anterior}$ y el nuevo candidato $x_{siguiente}$ pasa a ser x_{actual} . El punto más viejo se descarta ya que la secante solo tiene memoria para dos valores.

Salida por límite de iteraciones: Si el ciclo agota las 150 iteraciones sin lograr la convergencia, el método devuelve la mejor aproximación encontrada (el valor que arrojó menos valor absoluto de la función durante todo el proceso), asegurando así una salida numérica válida y evitando una multa por colapso del sistema.

El funcionamiento de metodoSecante.py también se puede representar a través del siguiente diagrama de flujo:

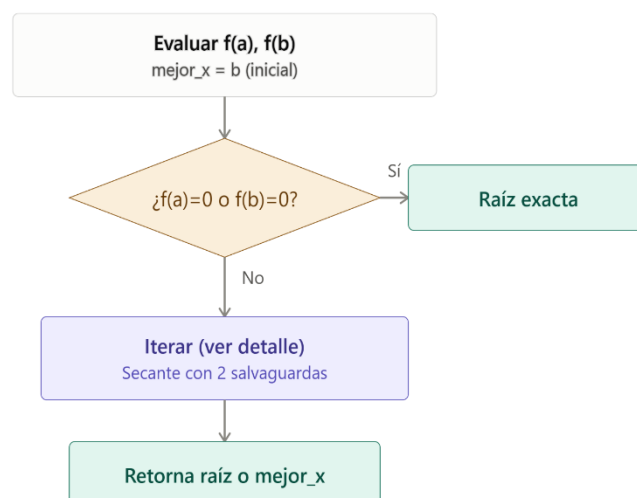


Ilustración 6: Diagrama de flujo de inicialización secante.

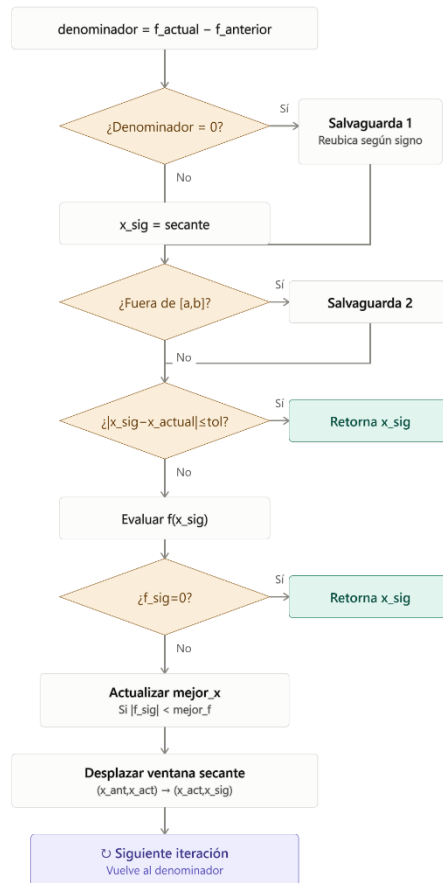


Ilustración 7: Diagrama de flujo de iteración secante.

Una vez probados, procedemos con los métodos combinados o “híbridos”, y de esta forma experimentar por nuestra propia cuenta:

Brent-Dekker (archivo metodoBrent.py):

Tipo de Método: Se trata de un método híbrido, combina de forma dinámica la velocidad de dos métodos abiertos (IQI y Secante) con la seguridad de un método cerrado (Bisección) cuando el paso propuesto no es matemáticamente confiable.

Validación de Encierro Inicial: Este método requiere que exista un cambio de signo en el intervalo inicial. Se evalúa $f(a)$ y $f(b)$. Si $fa \cdot fb > 0$ inmediatamente se arroja un ValueError porque matemáticamente este método no es válido.

Ordenamiento del Mejor Candidato: El algoritmo aplica una regla constante: el extremo b siempre debe ser la mejor estimación, debido a esto, si el valor absoluto de fa es menor al de fb el código intercambia sus posiciones.

El paso rápido (IQI o Secante): Al inicio del bucle, el código evalúa si los tres últimos puntos tienen imágenes distintas ($fa \neq fb$ and $fa \neq fc$), en este caso se calcula el candidato s trazando la parábola horizontal del método IQI. Si las imágenes no son distintas, se aplica la fórmula lineal de la Secanta para obtener s .

Las Condiciones de Seguridad de Brent: Antes de aceptar el candidato s calculado en el paso anterior, se evalúan 5 condiciones:

cond1: Verifica que el candidato no se haya salido del intervalo.

cond2 y *cond3:* Verifican que la longitud del paso no sea excesivamente larga respecto a saltos anteriores para evitar inestabilidad utilizando la variable *mflag*, que recuerda si el paso anterior fue una bisección.

cond4 y *cond5:* Evitan saltos muy pequeños que harían al algoritmo demasiado lento.

Si alguna de estas condiciones es verdadera (falla la seguridad) el código rechaza el candidato s calculado por IQI/Secante y fuerza el punto medio $s = \frac{a+b}{2}$ (bisección) encendiendo la bandera *mflag*=True.

Evaluación y Criterios de Parada: El nuevo punto s se evalúa de forma estándar (tipo="comun"), por lo que este método es muy barato porque no gasta puntos en derivadas. El ciclo se detiene exitosamente por tres motivos: si encuentra el cero exacto ($fs==0.0$), si el último paso es menor a la tolerancia ($abs(s - x \text{ anterior}) \leq tol$), o si el intervalo total colapsó a un tamaño menor a la tolerancia ($abs(b-a) \leq tol$).

Mantenimiento del Intervalo: Al finalizar la iteración se comprueba de qué lado quedó el cambio de signo ($fa \cdot fs < 0$) y actualiza los extremos a y b de la misma manera que lo hace Bisección para no perder nunca el encierro de la raíz.

Falla por Tiempo: Finalmente, si se agota el límite de iteraciones sin alcanzar las tolerancias, se arroja un RuntimeError explícito para informar al programa principal que el método fracasó.

El funcionamiento de `metodoBrent.py` también se puede representar a través del siguiente diagrama de flujo:

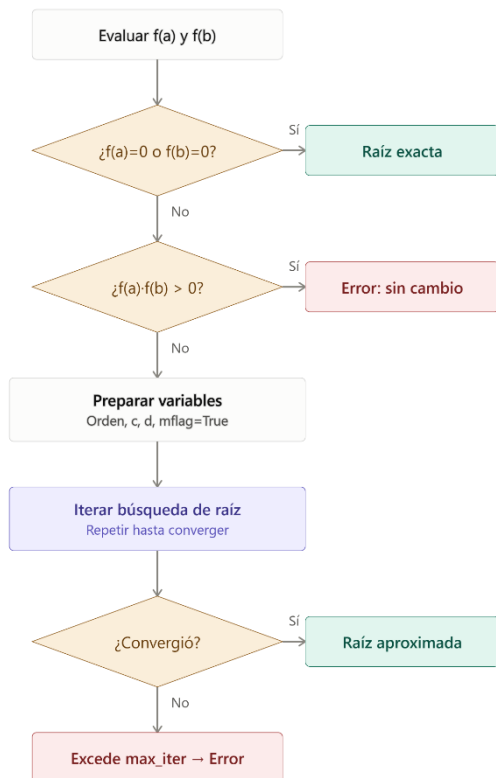


Ilustración 8: Diagrama de flujo inicialización de Brent-Dekker.

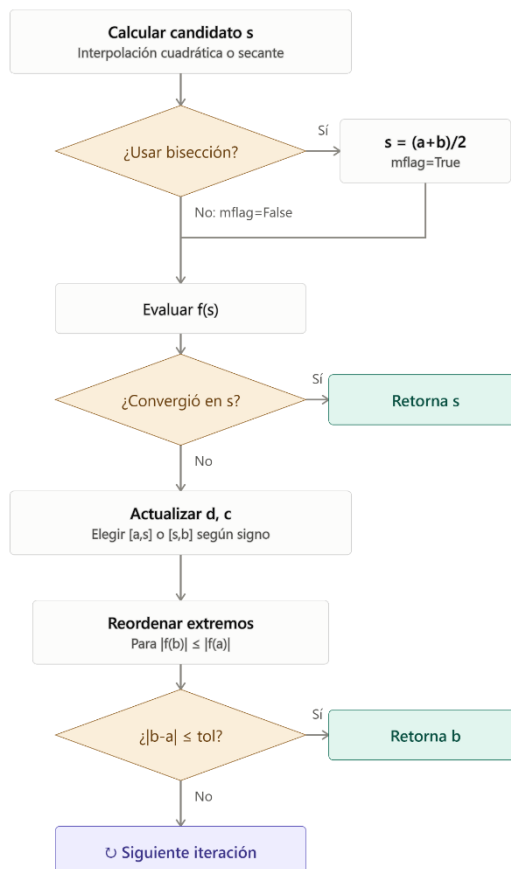


Ilustración 9: Diagrama de flujo de iteración Brent-Dekker.

Método Híbrido versión 1 (archivo metodoHibridoV01.py):

A diferencia del método de Brent que entrelaza diferentes técnicas en cada iteración, esta versión tiene una arquitectura de contingencia, la idea fue la siguiente:

- Si hay cambio de signo: Se aplica bisección pura para no arriesgarse a fallar.
- Si no hay cambio de signo: Se intenta un método rápido (Newton-Raphson).
- Si el método rápido falla o se sale de control: Se abandona ese camino y se usa un método abierto (Secante) pero forzándolo a mantenerse dentro de los límites del intervalo original.

Inicialización y Validaciones: Se establece el límite de iteraciones ($\text{max_iter}=150$) y se evalúan los dos extremos iniciales a y b con llamadas estándar (tipo="común"). Si alguno de los extremos es raíz exacta el programa termina y retorna dicho valor.

- **Fase 1 (Bisección):**

Activación: Se ejecuta única y exclusivamente si detecta un cambio de signo inicial ($f a \cdot f b < 0$).

Acción: Realiza el algoritmo de Bisección cortando el intervalo por la mitad en cada paso y evaluando $f(\text{raiz_estimada}, \text{tipo} = \text{"comun"})$.

Salida: Como el método es matemáticamente infalible bajo estas condiciones, si el programa entra en esta etapa iterará hasta que alcance la tolerancia exigida o devuelva la última estimación calculada en la última de las iteraciones.

- **Fase 2 (Newton-Raphson):**

Activación: Si no hubo cambio de signo el código pasa a esta fase. Primero elige como punto inicial el extremo del intervalo que arrojó el valor funcional más cercano a cero ($\text{abs}(f a) \leq \text{abs}(f b)$).

Acción: Se aplica el método y se calcula el candidato a raíz usando la fórmula clásica de Newton. Esta es la única fase que gasta puntaje del peaje calculando una derivada ($\text{tipo} = \text{"derivada"}$).

Corte de Emergencia: Si la derivada da cero, si el cálculo arroja un número no finito (not math.isfinite) o si el nuevo candidato se dispara fuera del intervalo original (a, b), el programa utiliza `break` para abortar la Fase 2 y continuar con la Fase 3.

Salida: Si se logra la estabilización y la distancia con el punto anterior es menor a la tolerancia (tol), devuelve el candidato exitosamente.

- **Fase 3 (Secante Acotada):**

Activación: Solo se llega a esta fase si la Fase 1 no aplicó y la Fase 2 falló o se volvió inestable. Inicia tomando como semillas los extremos originales a y b .

Acción: Aplica la fórmula iterativa de la Secante.

Acotación de seguridad: Como la Secante también puede arrojar valores fuera del intervalo se implementó una “capa de seguridad geométrica”: Si el nuevo candidato calculado se escapa de los límites permitidos ($\text{candidato} < a$ o $\text{candidato} > b$), el código lo descarta y lo reemplaza a la fuerza por el punto medio exacto $\frac{a+b}{2}$.

Salida: Al igual que el resto, se evalúa el punto con $\text{tipo} = \text{"comun"}$ y se detiene si se alcanza la exactitud o la tolerancia. Si todo falla y se acaban las iteraciones,

la función devuelve la mejor raíz estimada que haya logrado conseguir en todo el proceso.

El funcionamiento de metodoHibridoV01.py también se puede representar a través del siguiente diagrama de flujo (el de bisección ya salió anteriormente):

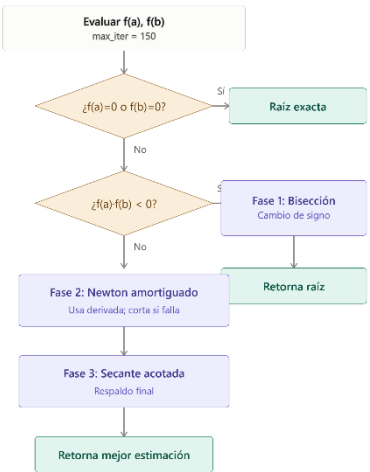


Ilustración 10: Diagrama de flujo general del método híbrido 1.

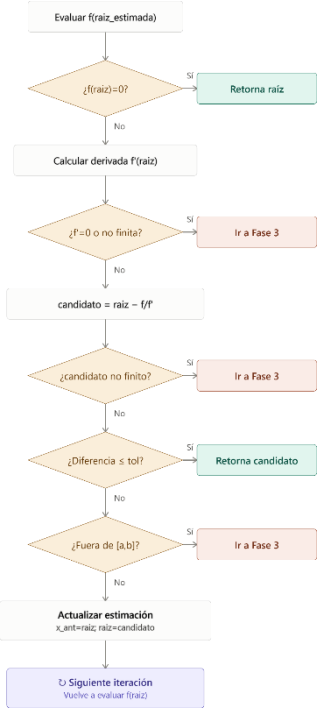


Ilustración 11: Diagrama de flujo de método híbrido 1, newton amortiguado.

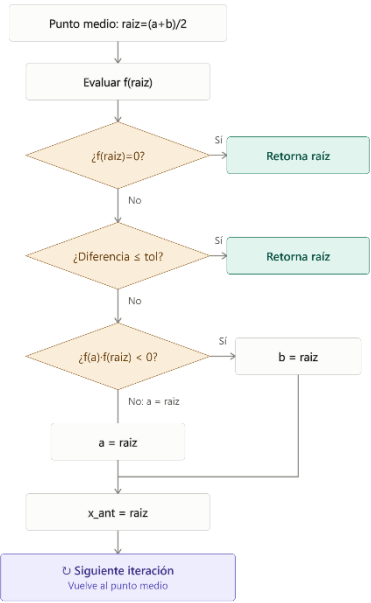


Ilustración 12: Diagrama de flujo de método híbrido 1, secante acotada.

Método Híbrido Version 2 (archivo metodoHibridoV02.py):

La idea con esta versión fue simplificar la lógica a solo dos ramas principales, dependiendo exclusivamente de si había o no cambio de signo en el intervalo inicial. También se implementó un sistema de “memoria”. En lugar de confiar únicamente en el último punto calculado, el algoritmo guarda constantemente cuál fue la mejor aproximación de toda la ejecución. Adicionalmente se integró Newton dentro de las ramas para que se ejecute antes de que el algoritmo suba penalizaciones por Impuesto a la Ráfaga Monótona, es decir, antes de llegar a las 5 llamadas consecutivas tipo=”comun”. Al ejecutar una derivada luego de la cuarta llamada tipo=”comun” se rompe la racha y los contadores de penalización.

Inicialización y Estado:

Se define el límite de iteraciones ($\text{max_iter}=150$).

Se declaran variables de estado clave: contador ($\text{evaluaciones_comunes}=0$), interruptor ($\text{newton_usado}=\text{False}$) y la memoria del mejor resultado (mejor_x y mejor_abs_f),

Se evalúan los extremos a y b directamente, se suma 2 al contador de evaluaciones comunes y se registra cuál de los dos extremos está más cerca del cero para guardarlo en la memoria.

- **Fase 1 (Rama Segura/Con cambio de signo):**

Activación: Si $f(a) \cdot f(b) < 0$.

Acción: Abandonamos la Bisección pura por su lentitud, en su lugar, se calcula un discriminante con los extremos para construir una aproximación mucho más veloz tipo Ridders.

En el Método de Ridders, se evalúa la función en los extremos del intervalo, $f(a)$ y $f(b)$, y en el punto medio, $f(c)$. El discriminante que calcula Ridders es el siguiente: $\sqrt{f(c)^2 - f(a)f(b)}$.

Salvaguarda: Si el discriminante da negativo (que generaría error matemático al calcularse su raíz cuadrada) o arroja un valor no finito se reemplaza el candidato por el punto medio $\frac{a+b}{2}$.

Mantenimiento: Conserva el cambio de signo de la misma manera que Bisección, asegurando que la raíz siga dentro del intervalo.

- **Fase 2 (Rama Abierta/Sin cambio de signo):**

Activación: Si los signos son iguales, comienza aplicando el método de Secante Acotada.

Acción: Se calcula el candidato trazando la recta secante con los últimos dos puntos y avanza actualizando esos puntos.

Salvaguarda: Si el cálculo arroja una división por cero, un valor no finito o el candidato escapa del intervalo original $[a, b]$, lo reemplaza por el punto medio del intervalo para forzar la estabilización.

- **El Refuerzo de Newton (presente en ambas fases):**

Sin importar la fase, siempre se tiene en cuenta el contador de llamadas. Si ya se hicieron 4 o más llamadas comunes (`evaluaciones_comunes >= 4`) y Newton todavía no ha sido ejecutado (`newton_usado=False`) el algoritmo interrumpe momentáneamente el flujo normal.

Se calcula una derivada (`tipo="derivada"`) y se propone un candidato de Newton en el momento. Si este candidato es numéricamente válido y se encuentra dentro del intervalo $[a, b]$, se acepta como nueva estimación de la raíz, se enciende el interruptor (`newton_usado=True`) y se suma una evaluación común al comprobar raíz en este nuevo punto.

- **Salida de Seguridad:**

Como vimos anteriormente, el bucle se interrumpe si encuentra una raíz exacta o si la distancia iterativa satisface la tolerancia (`abs(raiz_estimada - x_anterior) <= tol`).

La diferencia con la versión anterior es que si se acaban las iteraciones la función no devuelve el último cálculo (ya que podría ser un salto erróneo), sino que retorna `mejor_x`, es decir, la aproximación histórica acumulada que demostró estar matemáticamente más cercana del cero a lo largo de toda la ejecución.

El funcionamiento de `metodoHibridoV02.py` también se puede representar a través del siguiente diagrama de flujo:

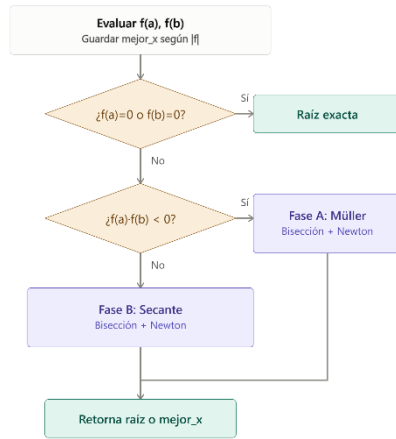


Ilustración 13: Diagrama de flujo general de método híbrido 2.

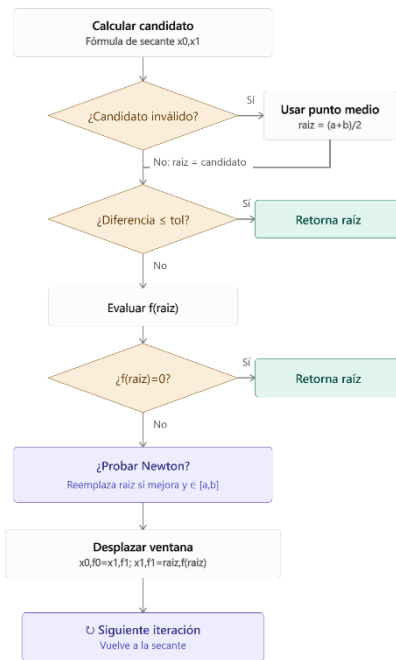


Ilustración 15: Diagrama de flujo de método híbrido 2, secante.

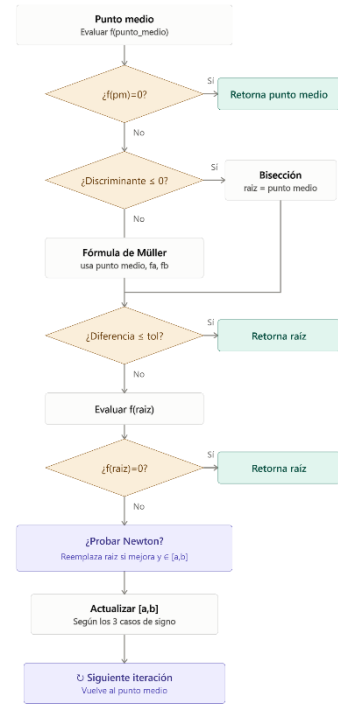


Ilustración 14: Diagrama de flujo de método híbrido 2, bisección + Newton.

Resultados:

En este apartado veremos los distintos resultados que nos arrojó el software que realizamos.

Método V01:

```
F1: ERROR -> El intervalo inicial no encierra una raíz (mismo signo).  
F1: +1500 por colapso del sistema  
F2: raiz = 0.750000000000 | f(raiz) = 0.000e+00 | puntaje = 8  
F3: raiz = 0.432876128108 | f(raiz) = 0.000e+00 | puntaje = 33  
F4: raiz = 0.228793221460 | f(raiz) = -3.747e-16 | puntaje = 31  
F5: raiz = 1.132267725273 | f(raiz) = 2.220e-16 | puntaje = 27  
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 30  
F7: raiz = 1.151292546497 | f(raiz) = -2.033e-20 | puntaje = 33  
F8: raiz = 2.046875000000 | f(raiz) = 1.940e+00 | puntaje = 1018  
F8: +1000 por raíz incorrecta (error = 4.014e-02)  
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 27  
F10: raiz = 5.000416305498 | f(raiz) = 3.563e-15 | puntaje = 31  
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 27  
F12: raiz = 2.031250000000 | f(raiz) = 3.150e-01 | puntaje = 1965  
F12: +1500 por colapso del sistema  
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 31  
F14: raiz = 2.019989186475 | f(raiz) = 8.882e-15 | puntaje = 31  
F15: raiz = 0.499737666569 | f(raiz) = 1.184e-16 | puntaje = 31  
F16: raiz = 1875.000000000000 | f(raiz) = 1.469e+00 | puntaje = 1018  
F16: +1000 por raíz incorrecta (error = 6.244e+02)  
F17: raiz = 9999.000000001783 | f(raiz) = 0.000e+00 | puntaje = 34  
PUNTAJE FINAL = 5877
```

Este método fue el primero que hicimos donde buscamos utilizar la bisección primero, para luego aplicar el de NR, este es el método mencionado anteriormente donde al aplicar la bisección, le pasamos como argumento una tolerancia distinta a la correcta, por lo que esta implementación no fue válida. Sin embargo, mostramos acá el resultado para mostrar nuestro desarrollo durante el trabajo práctico.

Método V02:

```
F1: raiz = 1.500000000000 | f(raiz) = 0.000e+00 | puntaje = 3  
F2: raiz = 0.750000000000 | f(raiz) = 0.000e+00 | puntaje = 16  
F3: raiz = 0.432876128108 | f(raiz) = 0.000e+00 | puntaje = 41  
F4: raiz = 0.228793221460 | f(raiz) = -3.747e-16 | puntaje = 39  
F5: raiz = 1.132267725273 | f(raiz) = 2.220e-16 | puntaje = 35  
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 38  
F7: raiz = 1.151292546497 | f(raiz) = -2.033e-20 | puntaje = 41  
F8: raiz = 2.046875000000 | f(raiz) = 1.940e+00 | puntaje = 1026  
F8: +1000 por raíz incorrecta (error = 4.014e-02)  
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 35  
F10: raiz = 5.000416305498 | f(raiz) = 3.563e-15 | puntaje = 39  
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 35  
F12: raiz = 2.031250000000 | f(raiz) = 3.150e-01 | puntaje = 1973  
F12: +1500 por colapso del sistema  
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 39  
F14: raiz = 2.019989186475 | f(raiz) = 8.882e-15 | puntaje = 39  
F15: raiz = 0.499737666569 | f(raiz) = 1.184e-16 | puntaje = 39  
F16: raiz = 1875.000000000000 | f(raiz) = 1.469e+00 | puntaje = 1026  
F16: +1000 por raíz incorrecta (error = 6.244e+02)  
F17: raiz = 9999.000000001783 | f(raiz) = 0.000e+00 | puntaje = 42  
PUNTAJE FINAL = 4506
```

Este método 2, es la misma implementación del 1, pero que simplemente agregamos una validación más donde, si los puntos extremos de la función son ambos positivos o negativos, se aplica directamente Newton-Raphson. Aplica lo dicho en el apartado anterior para este.

Método Secante:

```
F1: raiz = 1.500000000000 | f(raiz) = 0.000e+00 | puntaje = 3
F2: raiz = 0.750000029341 | f(raiz) = 1.872e-53 | puntaje = 596
F3: raiz = 0.000000000001 | f(raiz) = -1.000e-04 | puntaje = 1112
F3: +1000 por raiz incorrecta (error = 4.329e-01)
F4: raiz = 0.228793221460 | f(raiz) = -3.747e-16 | puntaje = 28
F5: raiz = 1.132267725273 | f(raiz) = -4.441e-16 | puntaje = 28
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 48
F7: raiz = 1.151292546497 | f(raiz) = -2.033e-20 | puntaje = 160
F8: raiz = 2.500000000000 | f(raiz) = 4.307e+00 | puntaje = 1004
F8: +1000 por raiz incorrecta (error = 4.933e-01)
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 32
F10: raiz = 5.000416305498 | f(raiz) = 3.563e-15 | puntaje = 32
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 24
F12: raiz = 1.897124736376 | f(raiz) = -4.686e-01 | puntaje = 1596
F12: +1000 por raiz incorrecta (error = 1.029e-01)
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 24
F14: raiz = 2.019989186475 | f(raiz) = 8.882e-15 | puntaje = 28
F15: raiz = 0.499737666569 | f(raiz) = -1.509e-16 | puntaje = 20
F16: raiz = 0.000000000000 | f(raiz) = -1.670e+00 | puntaje = 1060
F16: +1000 por raiz incorrecta (error = 1.251e+03)
F17: raiz = 198109.364400699269 | f(raiz) = 9.495e-05 | puntaje = 1596
F17: +1000 por raiz incorrecta (error = 1.881e+05)
PUNTAJE FINAL = 7391
```

Este resultado nos dio nuestra implementación del método secante. Para esta implementación tuvimos muchas dificultades porque el método es abierto, entonces no conseguimos hacer que se mantenga en el intervalo que se envía como argumento. Finalmente resolvimos que al implementar el regula falsi, ya que se trata del método secante adaptado para un intervalo cerrado, podríamos apreciar mejor el funcionamiento de este método.

Método Regula Falsi:

```
F1: ERROR -> El intervalo no contiene un cambio de signo
F1: +1500 por colapso del sistema
F2: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F2: +1500 por colapso del sistema
F3: raiz = 0.432876128108 | f(raiz) = 0.000e+00 | puntaje = 96
F4: raiz = 0.228793221460 | f(raiz) = -3.747e-16 | puntaje = 28
F5: raiz = 1.132267725273 | f(raiz) = 2.220e-16 | puntaje = 40
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 56
F7: raiz = 1.151292546497 | f(raiz) = 8.674e-19 | puntaje = 100
F8: raiz = 2.006737946999 | f(raiz) = -3.197e-14 | puntaje = 60
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 36
F10: raiz = 5.000416305498 | f(raiz) = 2.492e-14 | puntaje = 32
F11: raiz = 2.238653829835 | f(raiz) = -1.776e-15 | puntaje = 28
F12: raiz = 2.000000000000 | f(raiz) = -5.477e-05 | puntaje = 144
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 32
F14: raiz = 2.019989186475 | f(raiz) = -7.105e-15 | puntaje = 32
F15: raiz = 0.499737666569 | f(raiz) = 3.011e-13 | puntaje = 24
F16: raiz = 1250.550334672085 | f(raiz) = -4.045e-14 | puntaje = 76
F17: raiz = 9999.000000002499 | f(raiz) = 0.000e+00 | puntaje = 100
PUNTAJE FINAL = 4482
```

Ahora si conseguimos un resultado para casi todas las funciones, exceptuando F1, dado que no cumple con la condición de $f(a) \cdot f(b) > 0$, ni F2, ya que esta es una de las raíces que, según se verá en los otros resultados y en la conclusión, es de las que le cuesta más, según el método, encontrar.

Método IQI:

```
F1: raiz = 1.500000000000 | f(raiz) = 0.000e+00 | puntaje = 3
F2: raiz = 0.750000000000 | f(raiz) = 0.000e+00 | puntaje = 8
F3: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F3: +1500 por colapso del sistema
F4: raiz = 0.228793221460 | f(raiz) = 9.437e-16 | puntaje = 24
F5: raiz = 1.132267725273 | f(raiz) = 2.220e-16 | puntaje = 24
F6: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F6: +1500 por colapso del sistema
F7: ERROR -> math range error
F7: +1500 por colapso del sistema
F8: ERROR -> expected a positive input, got -2.1662019244229094
F8: +1500 por colapso del sistema
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 40
F10: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F10: +1500 por colapso del sistema
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 24
F12: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F12: +1500 por colapso del sistema
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 20
F14: raiz = 2.019989186475 | f(raiz) = -7.105e-15 | puntaje = 24
F15: raiz = 0.499737666569 | f(raiz) = 3.881e-16 | puntaje = 28
F16: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F16: +1500 por colapso del sistema
F17: ERROR -> No se alcanzó la tolerancia en el máximo de iteraciones
F17: +1500 por colapso del sistema
PUNTAJE FINAL = 15807
```

El principal problema que encontramos con este método fue que debido a la forma que toman las funciones propuestas (en especial las muy planas), no consigue encontrar la raíz dentro del intervalo con el número de iteraciones propuesto. Además, al tratarse de un método abierto, se da en algunos casos como el F7 y F8 que se vaya del dominio deseado y de errores.

Método Brent:

```
F1: ERROR -> El intervalo no contiene un cambio de signo
F1: +1500 por colapso del sistema
F2: raiz = 0.749999525901 | f(raiz) = -5.384e-45 | puntaje = 1120
F2: +1000 por raiz incorrecta (error = 4.741e-07)
F3: raiz = 0.432876128108 | f(raiz) = 1.355e-19 | puntaje = 44
F4: raiz = 0.228793221460 | f(raiz) = -3.747e-16 | puntaje = 28
F5: raiz = 1.132267725272 | f(raiz) = -2.977e-12 | puntaje = 52
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 32
F7: raiz = 1.151292546497 | f(raiz) = 1.525e-20 | puntaje = 44
F8: raiz = 2.006737946999 | f(raiz) = -3.197e-14 | puntaje = 44
F9: raiz = 1.231352007355 | f(raiz) = 2.776e-17 | puntaje = 36
F10: raiz = 5.000416305498 | f(raiz) = -1.206e-11 | puntaje = 40
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 20
F12: raiz = 2.000000000000 | f(raiz) = 6.329e-05 | puntaje = 128
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 24
F14: raiz = 2.019989186475 | f(raiz) = 8.882e-15 | puntaje = 32
F15: raiz = 0.499737666569 | f(raiz) = -1.509e-16 | puntaje = 20
F16: raiz = 1250.550334672087 | f(raiz) = 1.085e-12 | puntaje = 84
F17: raiz = 9998.999999996782 | f(raiz) = 0.000e+00 | puntaje = 56
PUNTAJE FINAL = 3306
```

Con esta implementación del método Brent conseguimos el resultado esperado para casi todas las funciones. En el caso de la función 1, el problema es el mismo que vimos en casos anteriores, donde cumple con $f(a) \cdot f(b) > 0$, por ende, el método no puede resolverlo. Mientras que por el caso de F2, lo que sucede es que, como la condición de parada es $|x_k - x_{k-1}| \leq \text{tol}$, al ser F2 una función extremadamente plana en el intervalo a analizar, en un momento el $x_k - x_{k-1}$ se acerca tanto al cero que salta como un 0 para nuestro código, pero no es realmente punto buscado de la raíz.

Método Híbrido V01:

```
F1: raiz = 1.500000000000 | f(raiz) = 0.000e+00 | puntaje = 6
F2: raiz = 0.750000000000 | f(raiz) = 0.000e+00 | puntaje = 4
F3: raiz = 0.432876128108 | f(raiz) = -1.700e-15 | puntaje = 156
F4: raiz = 0.228793221461 | f(raiz) = 4.446e-12 | puntaje = 152
F5: raiz = 1.132267725273 | f(raiz) = -7.727e-14 | puntaje = 152
F6: raiz = 1.059668473122 | f(raiz) = -2.993e-10 | puntaje = 160
F7: raiz = 1.151292546497 | f(raiz) = 2.890e-17 | puntaje = 160
F8: raiz = 2.006737946999 | f(raiz) = 2.271e-11 | puntaje = 156
F9: raiz = 1.231352007354 | f(raiz) = -1.395e-12 | puntaje = 152
F10: raiz = 5.000416305499 | f(raiz) = 1.151e-11 | puntaje = 156
F11: raiz = 2.238653829834 | f(raiz) = -2.187e-12 | puntaje = 160
F12: raiz = 2.000000000000 | f(raiz) = -6.104e-05 | puntaje = 164
F13: raiz = 0.626789006273 | f(raiz) = -1.171e-13 | puntaje = 152
F14: raiz = 2.019989186474 | f(raiz) = -2.158e-11 | puntaje = 152
F15: raiz = 0.499737666568 | f(raiz) = 2.390e-12 | puntaje = 152
F16: raiz = 1250.550334672085 | f(raiz) = -2.656e-13 | puntaje = 216
F17: raiz = 9999.000000004798 | f(raiz) = 0.000e+00 | puntaje = 176
PUNTAJE FINAL = 2426
```

Este método es el descrito anteriormente con las 3 fases donde analiza según el caso, qué método utilizar, y logramos un resultado exitoso para todas las funciones, pero con un puntaje relativamente alto.

Método Híbrido V02:

```
F1: raiz = 1.500000000000 | f(raiz) = 0.000e+00 | puntaje = 3
F2: raiz = 0.7500000000293 | f(raiz) = 1.859e-67 | puntaje = 126
F3: raiz = 0.432876128108 | f(raiz) = 0.000e+00 | puntaje = 33
F4: raiz = 0.228793221460 | f(raiz) = 4.996e-16 | puntaje = 31
F5: raiz = 1.132267725273 | f(raiz) = 2.220e-16 | puntaje = 31
F6: raiz = 1.059668473122 | f(raiz) = 0.000e+00 | puntaje = 32
F7: raiz = 1.151292546497 | f(raiz) = -3.896e-20 | puntaje = 38
F8: raiz = 2.006737946999 | f(raiz) = 3.464e-14 | puntaje = 55
F9: raiz = 1.231352007355 | f(raiz) = 3.192e-16 | puntaje = 31
F10: raiz = 5.000416305498 | f(raiz) = 3.563e-15 | puntaje = 33
F11: raiz = 2.238653829835 | f(raiz) = 1.776e-15 | puntaje = 31
F12: raiz = 2.000000000000 | f(raiz) = 4.661e-05 | puntaje = 112
F13: raiz = 0.626789006273 | f(raiz) = 0.000e+00 | puntaje = 31
F14: raiz = 2.019989186475 | f(raiz) = 8.882e-15 | puntaje = 31
F15: raiz = 0.499737666569 | f(raiz) = -1.509e-16 | puntaje = 33
F16: raiz = 1250.550334672085 | f(raiz) = -4.045e-14 | puntaje = 77
F17: raiz = 9998.999999996333 | f(raiz) = 0.000e+00 | puntaje = 64
PUNTAJE FINAL = 792
```

Finalmente, este fue el último método que realizamos, que nos pareció que conseguimos un puntaje muy bajo y también encontró todas las raíces. También nos dio un puntaje bastante bajo por cómo está hecho el método en sí, alternando cada 4 llamadas normales con una derivada. Los únicos casos en que obtuvimos un puntaje relativamente alto fue el F2 y el F12, que requirieron un

alto número de iteraciones para encontrar las raíces, en el caso de F2 podemos ver que al programa le cuesta mucho llegar al valor de resultado, iterando múltiples veces hasta terminar con un valor en el rango de 10^{-67} , mientras que por F12 sucede que la derivada en ese punto es infinita, entonces al programa le cuesta más encontrar el punto exacto que cumpla con la condición de parada.

Conclusión:

Al probar los distintos métodos, pudimos entender mejor cuales eran las fortalezas y debilidades de cada uno a la hora de encontrar raíces para distintos tipos de funciones, entendiendo cómo podríamos hacer para aprovechar sus funcionamientos para complementarlos y crear así un método híbrido que pueda ser utilizado para funciones muy variadas entre sí y con comportamientos diferentes en sus respectivos rangos.

Utilizando estos métodos investigados, conseguimos crear el método híbrido 2 que encuentra los resultados de todas las raíces con los rangos pedidos por la cátedra, y también funciona muy bien por la combinación de métodos y por la interrupción de la ráfaga monótona por la validación de newton que corta esa ráfaga de llamadas comunes con una del tipo derivada.